

# Project Brief: Custom Markdown → HTML Syntax Engine

*Written so that someone with zero programming background can follow it start to finish*

*G18 – Ojashvi Panwar, Sarthak Saxena, Shrishti Garg and Vanshika Agarwal*

## Background: What We're Actually Building

**HTML** (HyperText Markup Language) is the language every web page is written in. A web browser doesn't understand plain paragraphs of text on its own — it understands text wrapped in **tags**, which are instructions in angle brackets that tell the browser how to display something. For example, `<h1>Welcome</h1>` tells the browser "display the word 'Welcome' as a big heading," and `<strong>important</strong>` tells it "display the word 'important' in bold." Every tag that opens, like `<h1>`, has a matching closing tag with a forward slash, like `</h1>`, and everything in between is what gets that formatting.

**Markdown** is a much simpler way for a human to write formatted text without typing out all those tags. Instead of typing `<strong>bold</strong>`, you just type `**bold**`. Instead of `<h1>Welcome</h1>`, you type `# Welcome`. It's the format used by apps like Notion, Slack, GitHub, and Discord to let people write formatted text quickly using plain symbols like `#`, `*`, and `-` instead of clicking bold/italic buttons.

Our project is to build a program — from scratch, without using any pre-built Markdown tool — that **reads Markdown-style text and automatically converts it into the equivalent HTML**. In other words, we are writing the translator ourselves.

## Key Terms, Explained From Scratch

Before describing the plan, here is every important term used later in this document, explained in plain language.

### Python

Python is the programming language we're using to build this. A programming language is simply a set of precise, typed instructions a computer can follow — a Python program is just a text file full of these instructions, which the computer reads from top to bottom (with some exceptions we'll explain) and executes. We chose Python because its instructions read close to plain English, which makes it a good fit for a team that includes people newer to programming.

**Standard library** refers to the built-in tools that come packaged with Python automatically — no extra downloads or installs needed. Our project uses **only** these built-in tools (specifically two of them, called `re` and `html`, explained further below), which means anyone on the team can run the project the moment they install Python — nothing else to set up.

### Script / Program / File

A **script** or **program** is just a text file with a `.py` extension, containing Python instructions. When you "run" it, Python reads the file and carries out the instructions in order, one line at a time, from the top of the file down.

### Function

A **function** is a named, reusable block of instructions that performs one specific job. Think of it like a labeled recipe card: you give it some ingredients (called **inputs** or **arguments**), it follows its steps, and it hands back a result (called its **return value**). Instead of rewriting the same steps every time you need them, you just "call" the function by name. For example, we will write a function whose whole job is "take one line of plain text and turn any `**bold**` inside it into `<strong>bold</strong>`." Anywhere in the program that needs that done, it just calls that one function rather than repeating the logic.

### Class and Object

A **class** is a blueprint for bundling related data and functions together under one name. An **object** is one actual instance built from that blueprint. As an everyday analogy: "Car" as a general concept (with an engine, wheels, a colour) is like a class; your specific car in your driveway is an object made from that blueprint. In our project, we'll create a class for each kind of Markdown element — one for "Heading," one for "List," one for "Blockquote" — each bundling together the logic for "how do I recognize this kind of line" and "how do I convert it to HTML."

## Loop

A **loop** is an instruction that repeats a block of steps automatically, instead of the programmer writing the same steps out by hand over and over. Our program will use a loop to walk through a document one line at a time, from the first line to the last, applying the same checks to each line.

## Regular Expressions ("Regex")

A **regular expression**, almost always shortened to **regex**, is a mini pattern-matching language used to search for and replace pieces of text that follow a certain shape, rather than one exact fixed phrase. For example, instead of only being able to find the exact text "bold", a regex pattern can say "find any piece of text that sits between two pairs of asterisks," and it will correctly catch **bold**, **important**, or **anything else** — anything matching that shape. Python's built-in `re` module (part of the standard library mentioned above) is what lets us write and apply these patterns. Regex is extremely good at this kind of small, local, one-piece-of-text-at-a-time matching, which is exactly the job we'll give it (explained more under "Why Two Different Techniques," below).

## Parsing

**Parsing** simply means reading through raw text and figuring out its structure and meaning, rather than treating it as one undifferentiated blob of characters. When our program looks at a line of text and decides "this is a heading" or "this is the third item of a list," that decision-making process is called parsing.

## State / State Tracker

**State** just means "the current condition of something, which can change over time and needs to be remembered." A simple everyday example: a traffic light has a state — it is currently red, yellow, or green — and what it does next depends on what its current state is. In our program, as we read a document line by line, we need to remember things like "am I currently in the middle of reading a bulleted list, or not?" That remembered condition is the **state**, and the piece of code responsible for tracking it and updating it as we read further lines is what we call the **state tracker**. This is different from regex, which has no memory at all — each regex check only ever looks at the one small piece of text directly in front of it and forgets everything else, which is exactly why we need something else (the state tracker) to handle structure that spans multiple lines.

## Recursion

**Recursion** is when a function calls itself, in order to solve a smaller version of the same problem it was already trying to solve. A simple analogy: imagine a set of Russian nesting dolls, and your task is "open a doll, and if there's another doll inside, do the exact same thing to it." You don't need a different instruction for each layer — the same one instruction, applied again, handles any number of layers. We use this idea because Markdown content can nest inside itself (a bulleted list can appear inside a quoted block, which could itself appear inside another list). Rather than writing separate code for every possible combination, we let our structure-reading logic call itself on any nested content it finds, exactly like the doll example.

## Escaping (for security)

**Escaping** text means converting characters that have a special meaning (like `<` and `>`, which normally start and end an HTML tag) into a harmless, safe form before they're inserted into the final HTML output. This matters because if someone typed actual HTML code — including something malicious, like a `<script>` tag designed to run harmful code in a browser — directly into their Markdown text, and we copied it into our output completely unprotected, a browser would treat it as a real, functioning tag rather than as plain text the user typed. Python's built-in `html` module handles this conversion automatically and safely for us.

## Rule / Rule Registry

A "rule," in this project, is simply one self-contained piece of logic that knows how to recognize and handle one kind of Markdown element — for instance, a "Heading rule" or a "List rule." A "registry" is just an organized list we keep all these rules in, so the program can check each candidate rule in turn against a line of text and ask, "do you recognize this?" This design means that adding support for a brand-new kind of Markdown element later is as simple as writing one new rule and adding it to the list — nothing else in the program needs to change.

## What the Finished Program Does

The finished program is a tool you run from the **command line** (a text-based way of interacting with your computer, as opposed to clicking icons) that takes one input file — a plain `.md` or `.txt` file containing Markdown — and produces one output file — a working `.html` file, ready to be opened directly in a web browser.

For example, given this as the input file:

```
# My Title
This is bold and this is italic.
- item one
- item two

> A quote with a list inside:
> - point one
> - point two
```

The program produces this as the output file:

```
<h1>My Title</h1>
<p>This is <strong>bold</strong> and this is <em>italic</em>.</p>
<ul>
  <li>item one</li>
  <li>item two</li>
</ul>
<blockquote>
<p>A quote with a list inside:</p>
<ul>
  <li>point one</li>
  <li>point two</li>
</ul>
</blockquote>
```

Notice the last part of the example: a bulleted list sitting *inside* a quoted block. This kind of nested combination is why the "recursion" idea explained above matters — it lets the program correctly handle structure inside structure, not just flat, simple cases.

What makes this project a good fit for a team assignment is that success is completely **binary and testable**: for any given input file, there is exactly one correct HTML output. We can write a list of sample inputs together with their expected correct outputs, run the program, and it either matches or it doesn't — there's no room for subjective disagreement about whether something "looks right."

## Why Two Different Techniques Are Used Together

Markdown formatting actually falls into two different categories that behave very differently, which is why we use two different tools rather than one:

- **Block-level structure** — things like headings, paragraphs, bulleted lists, and quoted blocks — depends on *multiple lines and what came before them*. Whether a line beginning with a dash is "the second item of a list I already started" depends entirely on the lines above it. This needs **memory**, so we handle it with the state tracker concept explained above.
- **Inline-level formatting** — bold text, italic text, links, inline code — only ever depends on the tiny piece of text directly around it, never on anything from earlier or later in the document. This needs no memory at all, which is exactly what **regex** (explained above) is well-suited for.

So the program's plan is: first use the state tracker to figure out the overall shape of the document — where each paragraph, list, and quote begins and ends — and only *after* that, run the small, local regex checks on the text found inside each of those pieces, to catch bold and italic formatting.

## How the Program Works, Step by Step

- **1. Read the file.** The program opens the input text file and splits it into a list of individual lines, so it can look at them one at a time.
- **2. Figure out structure (the state tracker).** Using a loop (explained above), the program walks through the lines one by one. For each line, it checks its list of rules (explained above) in order and asks, "does anyone recognize this line?" Whichever rule matches takes over, and — if it's a rule for something that can span several lines, like a list — keeps collecting further lines into a group until it reaches a blank line or a different kind of element, then hands control back to the loop.
- **3. Handle nesting (recursion).** If the content collected in step 2 might itself contain further Markdown structure — for example, the text inside a quoted block might itself contain a list — that inner content is sent right back through step 2 again, exactly as explained in the "Recursion" section above.
- **4. Make the text safe (escaping).** Before any of our own HTML tags get added, the plain text itself is run through Python's built-in safety conversion (explained above under "Escaping"), so that any angle brackets a user happened to type can't be mistaken for real, functioning HTML by a browser.
- **5. Handle inline formatting (regex).** Now that a piece of text is confirmed to be, say, "the contents of a paragraph," the program runs it through its list of regex patterns, in a deliberate order — bold text is checked before italic text, because checking italics first could accidentally cut a bold marker in half.
- **6. Wrap and save.** Each finished piece of text is wrapped in its correct HTML tags (the ones explained at the very start of this document) and the complete result is written out to a new HTML file, ready to open in a browser.

## Project Structure: What Each File Is For

The project will be organized into a folder containing the following files:

```
markdown_engine/
├── main.py           # entry point: reads file, runs engine, writes output
├── parser.py         # the state tracker + recursion logic
├── rules.py          # the list of block rules + the list of regex patterns
├── tests/
│   └── test_cases.py # sample inputs + their expected correct outputs
```

- `main.py` — the starting point of the program. This is the file you actually run. Its job is simple: open the input file, hand its contents to the logic in the other files, and save whatever comes back as the output file.
- `parser.py` — contains the state tracker: the loop that walks through the document's lines and figures out its overall structure, including the recursive step for nested content.
- `rules.py` — contains the organized list ("registry") of block rules (heading, list, blockquote, paragraph) and the ordered list of regex patterns for inline formatting (bold, italic, and so on).
- `tests/test_cases.py` — a separate file whose only purpose is to check that the program works correctly. It contains a series of small sample inputs paired with the exact HTML we expect the program to produce, using Python's built-in `unittest` tool (also part of the standard library) to automatically check whether the real output matches the expected output.

## Build Plan: The Order We'll Build This In

Rather than trying to build the whole thing at once, we'll build it in stages, each one working and testable before moving to the next:

- **Stage 1 — Headings only.** Get the entire pipeline working end-to-end for just one simple rule: turning `# Text` into `<h1>Text</h1>`. This proves that reading the file, processing it, and writing the output file all work correctly together, before any complexity is added.
- **Stage 2 — Inline formatting.** Add the regex patterns for bold and italic text, being careful about the order they're checked in.
- **Stage 3 — Structural elements (the state tracker).** The most involved stage: teaching the program to recognize and correctly group lists, paragraphs, and quoted blocks, including the recursive handling of nested content. It's

recommended that this stage be sketched out on paper — literally drawing out "what state am I in, and what makes me switch to a different state" — before any code is written for it.

- **Stage 4 — Extra features.** Things like code blocks (where formatting rules should be turned off, so code isn't accidentally styled), clickable links, and letting users type a backslash before a symbol to stop it from being treated as formatting.
- **Stage 5 — Testing.** Writing a thorough set of sample-input-and-expected-output pairs, covering both individual rules on their own and tricky combinations like formatting nested inside other formatting, so that any future change to the program can be checked against these tests to make sure nothing broke.

## Why We're Building It This Way

- It demonstrates real, from-scratch programming and text-processing skills, rather than relying on an existing tool to do the actual work for us
- The goal is completely clear and checkable — a given input always has exactly one correct output, so there's never ambiguity about whether something is "done"
- Because structure is handled by a state tracker that can call itself on nested content, the program can correctly handle combinations that nobody explicitly planned for in advance — not just the specific examples used in our tests
- New kinds of Markdown formatting can be added later just by writing one new self-contained rule, without needing to touch or risk breaking anything already working
- The four files map naturally onto four people, each with a clearly separate responsibility, so the team can work at the same time without getting in each other's way
- It's realistic to finish in stages — even stopping after Stage 1 or 2 leaves something working and demonstrable, and each further stage builds cleanly on the last